

CENTRO DE ENSEÑANZA TÉCNICA Y SUPERIOR



Programación orientada a objetos

Reporte técnico

Presentado por:

Fatima Sofia Vazquez Acosta

Juan Jose de Jesus Castro Guerrero

Mia Paola de la Cruz Hernandez

Luis David Vigueros Ramírez

Tijuana, B.C, 8 de Mayo de 2026

- Descripción General del Proyecto

ONE-WEEK es un simulador de citas (dating sim) que pone al jugador en el papel de un protagonista cuya misión es equilibrar su vida personal durante una semana crítica. A lo largo de la historia, el jugador deberá gestionar una nueva amistad con un personaje llamado Illeana sin descuidar la relación con su pareja actual.

El núcleo del juego reside en la toma de decisiones: cada opción elegida impacta variables de Felicidad y Confianza, las cuales determinan el nivel total de Amor. El juego culmina en uno de los cuatro finales posibles, calculados según el desempeño emocional del jugador.

- Especificación de Funcionalidades

-Elección de Prototipo de Protagonista:

Al inicio, el jugador selecciona entre cuatro arquetipos: NPC, Tóxico, Gymrat u Otaku. Cada uno posee atributos base únicos (Atractivo, Carisma y Lealtad). Estos atributos actúan como multiplicadores o modificadores en la lógica de impacto, haciendo que una misma decisión tenga resultados distintos dependiendo de quién sea el protagonista.

-Sistema de Escenas e Interacción:

El motor visual utiliza fondos dinámicos que cambian según el progreso narrativo. La interacción se realiza mediante un sistema de colisiones por ratón sobre botones integrados en la interfaz.

-Personalización:

Implementación de un sistema de entrada de texto para que el jugador nombre al protagonista, permitiendo una mayor inmersión mediante la concatenación de strings en los diálogos.

-Gestión de Regalos:

Mecánica de inventario al final de cada día donde la selección de objetos afecta directamente el estado de ánimo de la novia.

-Mecánica de "Game Over" Inmediato:

El sistema monitorea constantemente el nivel de Amor. Si este cae por debajo de un umbral crítico (60 puntos), se dispara automáticamente el "Final Malo", terminando la partida prematuramente.

- Estructura del Sistema

El proyecto está diseñado bajo el paradigma de Programación Orientada a Objetos (POO):

-Librería SFML: Gestiona el ciclo de vida de la ventana, el renderizado de texturas, la reproducción de audio (streaming para música y buffers para efectos) y la captura de eventos.

-Clase Novia: Funciona como el contenedor de estado del NPC principal, gestionando sus estadísticas y métodos de actualización de atributos.

Jerarquía de Clases (Jugador):

-Clase Base Jugador: Define la estructura común (Atractivo, Carisma, Lealtad) y el método virtual para aplicar impactos.

-Clases Derivadas (Otaku, Toxico, Gymrat, NPC): Implementan el polimorfismo, definiendo valores específicos y comportamientos únicos basados en su perfil.

Estructura de Datos:

-struct Escenario: Almacena la carga narrativa (pregunta, opciones de respuesta e impactos asociados).

-vector<Escenario>: Funciona como la base de datos dinámica del juego, permitiendo una navegación lineal y eficiente por la historia.

Control de Flujo (Main): Implementa una Máquina de Estados simple mediante una variable controladora (scene) que coordina qué lógica y qué gráficos procesar en cada iteración del bucle.

- Justificación de conceptos de programación.

Encapsulamiento: Utilizamos este paradigma para proteger ciertos atributos de la clase de la novia que solo pueden ser modificados por medio de otras funciones.

La abstracción la aplicamos al momento de crear a los personajes mediante las clases; esto para una mejor representación de un sistema comparado con la vida real.

Fácilmente podemos notar que las subclases de personajes heredan directamente de una clase de jugador base para darles ciertos atributos genéricos y, después, sí, cualidades específicas.

El polimorfismo lo utilizamos al momento de llamar ciertas funciones por medio de punteros, permitiendo manejar diferentes tipos de personajes, la sobrecarga de funciones, y el uso de polimorfismo dinámico en las subclases.

Este programa fue desarrollado de manera modular, separando responsabilidades entre las clases y generando mejores funciones que permitan facilitar el mantenimiento del mismo juego, aunque se continúa trabajando en la separación por medio de librerías.

Uso de sfml

Se utilizó SFML para el desarrollo de ONE WEEK porque el proyecto necesitaba una interfaz basada en botones, fondos, texto y música, además de manejar clics del mouse.

SFML permite trabajar con gráficos 2D de una manera sencilla; esta librería fue clave ya que el juego se basa en tomar decisiones a través de botones.

La parte de SFML está principalmente en el archivo main, mientras que la lógica del juego está separada en clases. Esto ayudó a mantener el código más organizado.

Para poder trabajar con esta librería nos encargamos de hacer pruebas, y de ver cómo funcionaba. Para los fondos solo cargamos texturas y se las asignamos a un

sprite. Siempre que cargábamos una textura, corríamos el programa para ver que todo funcionara correctamente. Además, para los botones, se crearon áreas invisibles, a las cuales por medio de un rectángulo se les asignaba el mismo tamaño y posición y, a prueba y error se iban ubicando en la zona requerida.

En general, SFML facilitó mucho el desarrollo porque permite manejar la parte visual e interactiva del juego sin necesidad de programar gráficos desde cero.

- Capturas de pantalla

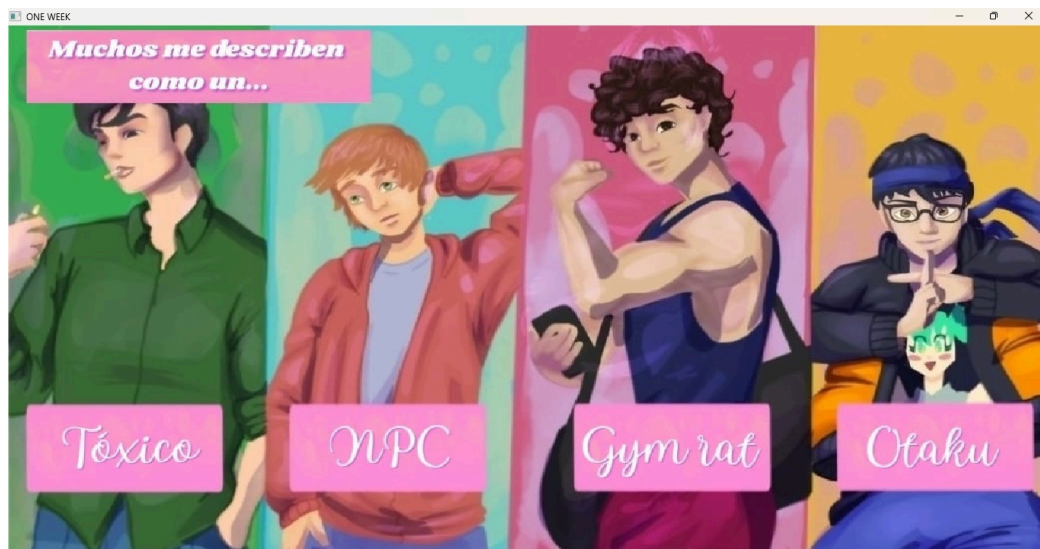


Figura 1.1 Selección de personajes



Figura 1.2 Escena de seleccion de toma de decisiones

```
C:\Users\Usuario\OneWeek x + v
Cargada correctamente: ../assets/fondos/Comidafeliz.png
Intentando cargar: ../assets/fondos/Sushimesero.png
Cargada correctamente: ../assets/fondos/Sushimesero.png
Intentando cargar: ../assets/fondos/Noviaescucha.png
Cargada correctamente: ../assets/fondos/Noviaescucha.png
Intentando cargar: ../assets/fondos/NoviaescuchaOP.png
Cargada correctamente: ../assets/fondos/NoviaescuchaOP.png
Intentando cargar: ../assets/fondos/Noviaenojada.png
Cargada correctamente: ../assets/fondos/Noviaenojada.png
Intentando cargar: ../assets/fondos/Noviatensa.png
Cargada correctamente: ../assets/fondos/Noviatensa.png
Intentando cargar: ../assets/fondos/Ileanallega.png
Cargada correctamente: ../assets/fondos/Ileanallega.png
Intentando cargar: ../assets/fondos/Ileanallegaop.png
Cargada correctamente: ../assets/fondos/Ileanallegaop.png
Intentando cargar: ../assets/fondos/Ileanaseva.png
Cargada correctamente: ../assets/fondos/Ileanaseva.png
Intentando cargar: ../assets/fondos/Pausa.jpeg
Cargada correctamente: ../assets/fondos/Pausa.jpeg
Escenario actual: 0
Scene actual: 7
Confianza: 105
Felicidad: 100
Amor: 72
Escenario actual: 1
Scene actual: 11
Confianza: 100
Felicidad: 100
Amor: 70
```

Figura 1.3 Registro en consola sobre el cambio de atributos



Figura 1.4 Escena de menu

- Conclusión técnica del equipo

Podemos concluir que la forma en la que se construyeron las clases fue lo clave para que el proyecto funcionara como esperábamos. Al final, todo el juego depende de cómo se comunicaban entre sí los distintos atributos, especialmente las decisiones del jugador y como estas afectan a la clase Novia.

En este proyecto nos enfrentamos a distintos problemas. Entre lo más destacado fue que los atributos del jugador no interactúan de la manera en la que nosotros esperábamos. A pesar de que fueran decisiones negativas, estas seguían incrementando el amor en vez de bajarlo. Pero con nuevas funciones y gracias al polimorfismo dinámico, se logró resolver la problemática. Aunque no era algo visto a simple vista, cada personaje podía comportarse de acuerdo a su propia lógica sin interferir con los demás. Llevando a que pudiéramos reutilizar el código, viéndose más ordenado y fácil de manejar.

Finalmente, el uso de la librería SFML le dio vida al proyecto. Permittiéndonos agregar elementos más visuales como imágenes, textos, música, efectos y botones. Esto hizo que nuestro juego no solo funcionara en la parte de la lógica, si no que también fuera más entretenido y divertido visualmente.

- Reflexión individual de cada integrante.

Luis David Vigueros Ramírez

Por medio de este proyecto desarrollé los conocimientos adquirimos como herencia, polimorfismo, que fueron los principales al momento de hacer las interacciones entre las clases, además de aprender a usar el polimorfismo dinámico para mejorar la interacción con funciones específicas de las subclases. Además de los tipos de relaciones entre clases, en nuestro caso, de asociación.

Fatima Sofia Vazquez Acosta

Este proyecto podría decir que fue de mis favoritos de hacer. Aprendí muchos conocimientos básicos de programación orientada a objetos: herencia, encapsulamiento, polimorfismo. También aprendí mucho más sobre lo visual y trabajar de frontend. Me gusto entender cómo se relaciona la lógica y se conectaba con las escenas y botones. Estoy muy feliz de cómo resultó el trabajo.

Juan José de Jesús Castro Guerrero

Este proyecto me ayudó a desarrollar mis conocimientos referentes a Github, ya que mi principal aportación consistió en ser el encargado de Github, yo creé el repositorio y contestaba dudas a mis compañeros sobre este tema. Mis mayores dificultades fueron entender los conceptos de Github y saber cómo usar los comandos de git desde Visual Studio. A su vez aprendí bastante sobre configuraciones de proyecto y uso de rutas relativas para poder hacerlo portable. Gracias a este proyecto aprendí cómo llevar una mejor comunicación con mi equipo y sobre cómo dividir un proyecto en diferentes apartados para que cada quien trabaje en una sección en específico.

Mia Paola De La Cruz Hernández

Este proyecto me ha sido de gran apoyo para el desarrollo de mis habilidades de trabajo en equipo, gestión de tiempo y conocimientos prácticos de POO.

Investigando, aclaré muchos conceptos de manera práctica, que me llevaron a entender mejor la teoría. A pesar de que en un inicio no modifique el código directamente, hacía lo posible por investigar y comprender lo que estaban realizando mis compañeros. Del mismo modo balancear el diseño y creación de los fondos junto con la programación de mis días correspondientes poco ser un pequeño reto. Sin embargo, estoy feliz con el desarrollo de mis conocimientos y la práctica que pude obtener.

Enlaces:

Para mayor información puede acceder a nuestra carpeta de drive o a nuestro repositorio de GitHub del proyecto.

Drive:

https://drive.google.com/drive/folders/1AwU2bhPxu9kfa0DjnxYRW8c4cFnhO1qn?usp=drive_link

GitHub: <https://github.com/JuanJoseCastro15/ONE-WEEK>